QueryAPIGate HOW-TO GUIDE 07 · QUERIES

Group related queries into a collection

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=07-group-queries-into-a-collection

Time: 5 minutes. **You'll end up with:** several saved queries filed under one collection name, and a scoped API key that can reach every query in it - present and future - without naming them one by one.

Put a query in a collection

Either at save time:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/queries -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "orders_count", "sql": "SELECT COUNT(*) AS n FROM orders",
  "connection_name": "shop", "author": "you", "description": "Order count",
  "collection": "reporting",
  "publish": true
}'
```

...or move an existing one in or out afterward, without creating a new version:

```shell
curl -X PATCH http://127.0.0.1:5000/api/v1/queries/products_count -H 'X-API-Key: demo-key' -H 'Content-
Type: application/json' \
  -d '{"collection": "reporting"}'
```

Every move is written to the audit log with exactly which existing keys gain or lose reach *because of this one move* - and the admin UI's **Move...** shows the same before you confirm it, so you find out before, not the hard way:

```shell
curl 'http://127.0.0.1:5000/api/v1/audit?action=move_query' -H 'X-API-Key: demo-key'
# {"items": [{"action": "move_query", "target": "products_count",
#             "changes": {"collection": {"from": null, "to": "reporting"},
#                         "keys_gaining_access": ["reporting-key"], "keys_losing_access": []}, ...}],
...}
```

Moving a query out again (`{"collection": null}`) works the same way, in reverse.

A collection name is 1-63 characters - lowercase letters, digits, `.`, `_` and `-`, starting with a letter or digit; anything else is rejected with a clear error, the same rule everywhere a collection name is accepted (saving, moving, granting a key, renaming).

Grant a key the whole collection

```shell
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "reporting-key", "connections": [], "collections": ["reporting"]
}'
```

`connections: []` here is deliberate, not an oversight: this key has **no** ad-hoc SQL access to any connection at all - `collections` is a purely additive grant, read-only, reaching exactly the saved queries currently filed under `reporting`, nothing more. (A key can hold `connections`, `queries` and `collections` grants all at once - they add reach together, never narrow each other.)

```shell
curl http://127.0.0.1:5000/q/orders_count -H 'X-API-Key: <reporting-key secret>'
# [{"n": 2}]
curl http://127.0.0.1:5000/q/products_count -H 'X-API-Key: <reporting-key secret>'
# {"error": "This API key is not permitted to use the connection 'shop'"} - not in the collection
```

The grant is live, not a snapshot

This is the actual point of a collection grant, verified for real: **a key's `collections` grant is re-checked from the query's current filing on every request, not frozen at the moment the key was created.** File a new query into `reporting` tomorrow, and every key already granted that collection reaches it immediately - no key edit, no redeploy.

```shell
# reporting-key already exists, already denied products_count above
curl -X PATCH http://127.0.0.1:5000/api/v1/queries/products_count -H 'X-API-Key: demo-key' \
  -H 'Content-Type: application/json' -d '{"collection": "reporting"}'

curl http://127.0.0.1:5000/q/products_count -H 'X-API-Key: <reporting-key secret>'
# [{"n": 1}] - reachable now, the key itself was never touched
```

The same is true in reverse - move a query out, and every key holding that collection loses it on the very next request.

Renaming a collection updates every key and role that reference it

```shell
curl -X PATCH http://127.0.0.1:5000/api/v1/collections/reporting -H 'X-API-Key: demo-key' -H 'Content-
Type: application/json' \
  -d '{"name": "sales-reporting"}'
# {"name": "sales-reporting",
#  "moved": {"queries": ["orders_count", "products_count"], "keys": ["reporting-key"], "roles": []}}
```

Verified: `reporting-key`'s own stored `collections` grant is rewritten from `["reporting"]` to `["sales-reporting"]` automatically - it keeps working with no manual edit. `merge: true` in the same request body folds the renamed collection into an already-existing one with the target name instead of failing (409) because it's taken.

Why use this instead of a `queries` grant listing each name

Both are valid, additive grants - the difference is what you're modeling:

- `queries: ["orders_count", "orders_total"]` - a fixed, curated list. Right for an external partner who should reach exactly these two endpoints and nothing you add later without a deliberate decision to add them by name.
- `collections: ["reporting"]` - an open membership. Right for an internal team's own key, or anything where "everything in this bucket" should mean everything in this bucket, including what gets added next week, without editing every key that should see it.

Next steps

- Set up your first scoped API key - the full grant-field reference (`connections`, `queries`, `collections`, and the rest) this guide only used one corner of.
- Verify who can reach what - the Access map shows every key's reach, collection grants included, at a glance.

READ NEXT

Cache a saved query's response

QUESTIONS OR PROBLEMS

Discord · GitHub issues

LATEST VERSION OF THIS PAGE

anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=07-group-queries-into-a-collection

FOLLOW QUERYAPIGATE GitHub X Discord Bluesky Mastodon YouTube Reddit

@queryapigate on X, Bluesky, Mastodon and YouTube · r/queryapigate on Reddit · github.com/AnanthaRajuC/QueryAPIGate

QueryAPIGate · one governed gate between your databases and everything that asks them for data · self-hosted, Apache-2.0 · © 2026 QueryAPIGate